FairLens — ML Bias Audit Toolkit

■■ ENDGAME TIER — Responsible AI Engineering

"Fairness Auditing, Bias Detection & Mitigation for Tabular ML Models"

■■ Endgame Project | Domain: Responsible AI + Fairness ML | Difficulty: Advanced | Timeline: 10 Weeks

■ Project Overview

Description	A production-grade fairness auditing library and dashboard for any scikit-learn-compatible tabular ML model. FairLens computes industry-standard fairness metrics (demographic parity, equalised odds, calibration by group), generates automated bias audit reports, suggests and applies mitigation strategies (reweighting, threshold adjustment, adversarial debiasing), and tracks fairness-accuracy tradeoffs across model versions.
Target Audience	Responsible AI engineers, ML researchers, policy-aligned tech teams, audit-focused ML practitioners
Domain	AI Fairness, Responsible ML, Bias Mitigation, XAI
Deliverable	Python fairness audit library (pip-installable) + Streamlit audit dashboard + PDF bias audit report generator
Why It Stands Out	Fairness and bias auditing is mandated by emerging AI regulation (EU AI Act, US Executive Order on AI). FairLens demonstrates you can build audit-ready ML systems — a rare and high-value skill in 2024+.

■■ Tech Stack

Language	Python 3.10+
Fairness Frameworks	Fairlearn (Microsoft), AIF360 (IBM), custom implementations

ML Libraries	scikit-learn, XGBoost, LightGBM
Statistical Tests	scipy (Chi-Square, Fisher Exact), bootstrapping for CIs
Explainability	SHAP (per-group feature importance)
Visualisation	Plotly, Matplotlib, Seaborn
Dashboard	Streamlit
Report Generation	ReportLab (PDF audit report)
Packaging	setuptools / pyproject.toml for pip-installable library
Dev Tools	Jupyter, VSCode, Git, pytest

■ MVP Features (Phase 1)

Fairness Metrics Engine

- ◆ Demographic Parity: P(Y_hat=1 | A=0) == P(Y_hat=1 | A=1)
- ◆ Equalised Odds: equal TPR and FPR across protected groups
- ◆ Equal Opportunity: equal true positive rate across groups
- ◆ Calibration by Group: predicted probabilities match actual outcomes per group
- ◆ Predictive Parity: equal positive predictive value (precision) across groups
- ◆ All metrics computed with 95% confidence intervals via bootstrapping

Intersectional Bias Analysis

- ◆ Single-attribute analysis: bias across gender, race, age bracket, region
- ◆ Intersectional analysis: bias across combined attributes (e.g. Black women, young men)
- ◆ Subgroup performance table: accuracy, F1, AUC-ROC per demographic slice
- ◆ Worst-slice identification: automatically surface the most disadvantaged subgroup

Bias Mitigation Strategies

- ◆ Pre-processing: sample reweighting to equalise group representation
- ◆ Pre-processing: resampling to balance target rates across groups
- ◆ In-processing: fairness-constrained training via Fairlearn (Exponentiated Gradient)
- ◆ Post-processing: threshold adjustment per group to equalise FPR/TPR
- ◆ Side-by-side fairness-accuracy tradeoff curve for each mitigation strategy

Audit Report Generator

- ◆ Model card: model description, training data, intended use, known limitations
- ◆ Per-metric fairness score with pass/fail against configurable thresholds
- ◆ Worst-performing subgroup highlighted with statistical significance test
- ◆ Mitigation recommendations ranked by fairness improvement vs accuracy cost
- ◆ Export as professional PDF audit report (ReportLab)

Streamlit Audit Dashboard

- ◆ Upload any sklearn model (.pkl) and test dataset (.csv) to run audit
- ◆ Select protected attribute(s) and reference group interactively
- ◆ Interactive fairness metric radarchart and subgroup performance heatmap
- ◆ Side-by-side before/after mitigation comparison dashboard

■ Development Roadmap

Week 1 Setup & Research	■ Set up Python environment, install Fairlearn, AIF360, SHAP, scipy ■ Study core fairness definitions: demographic parity, equalised odds, calibration ■ Prepare 3 benchmark datasets for auditing: Adult Income, COMPAS Recidivism, German Credit ■ Train 3 baseline models (one per dataset) to use as audit targets throughout
Week 2 Fairness Metrics Engine — Core Metrics	■ Implement demographic parity difference and ratio ■ Implement equalised odds difference (TPR gap + FPR gap) ■ Implement equal opportunity difference (TPR gap only) ■ Implement calibration-by-group (compare predicted prob vs actual outcome rate) ■ Validate all implementations against Fairlearn's reference implementations
Week 3 Confidence Intervals & Statistical Significance	■ Implement bootstrap resampling for 95% CIs on all fairness metrics ■ Add Fisher Exact test for demographic parity significance (small sample sizes) ■ Add Chi-Square test for independence between predictions and protected attributes ■ Build metric result object: value + CI lower + CI upper + p-value + significance flag
Week 4 Intersectional Analysis	■ Build subgroup slicer: generate all demographic subgroup combinations ■ Compute accuracy, F1, AUC-ROC, and all fairness metrics per subgroup ■ Build worst-slice detector: identify the subgroup with the lowest performance ■ Build subgroup performance heatmap (Plotly, groups on rows, metrics on columns)
Week 5 Bias Mitigation — Pre-Processing	■ Implement sample reweighting: compute instance weights to equalise group outcome rates ■ Implement resampling: oversample disadvantaged group to balance representation ■ Retrain baseline models with both strategies and measure fairness improvement ■ Plot fairness-accuracy tradeoff: scatter plot of F1 vs fairness metric before/after
Week 6 Bias Mitigation — In & Post Processing	■ Integrate Fairlearn Exponentiated Gradient for fairness-constrained training ■ Implement post-processing threshold adjustment per group (Hardt et al. method) ■ Run all 4 mitigation strategies on all 3 benchmark datasets ■ Build mitigation comparison leaderboard: rank strategies by fairness improvement per accuracy cost

Week 7 SHAP Per-Group Analysis	■ Compute SHAP values separately for each demographic group ■ Compare feature importance rankings across groups (do different features drive predictions per group?) ■ Identify features with high SHAP variance across groups (potential proxy discrimination sources) ■ Build group SHAP comparison chart: side-by-side beeswarm plots per group
Week 8 PDF Audit Report Generator	■ Design 5-section audit report: Model Card + Fairness Metrics + Subgroup Analysis + Mitigation + Recommendations ■ Build ReportLab PDF generator for all report sections ■ Add severity-rated recommendations (CRITICAL / WARNING / INFO) per metric ■ Validate audit report against EU AI Act high-risk system checklist items
Week 9 Streamlit Audit Dashboard	■ Build model + dataset upload interface (.pkl model, .csv test set) ■ Build protected attribute and reference group selector ■ Integrate fairness metric radarchart (Plotly) — all metrics in one view ■ Build before/after mitigation comparison panel with strategy selector
Week 10 Packaging, Testing & Docs	■ Package FairLens as pip-installable library with pyproject.toml ■ Write comprehensive pytest suite (unit tests for all metric implementations) ■ Validate correctness on COMPAS and Adult Income against published audit results ■ Publish to GitHub with full documentation, usage examples, and API reference ■ Write README with fairness definitions, usage guide, and EU AI Act context

■ Phase 2 — Future Enhancements

- ◆ NLP fairness auditing: extend FairLens to text classification and sentiment models
- ◆ Image model fairness: audit face recognition and CV models for demographic bias
- ◆ Longitudinal fairness tracking: monitor fairness metrics as models are retrained over time
- ◆ Regulatory compliance report templates (EU AI Act, US NIST AI RMF, ISO 42001)
- ◆ Automated data auditing: audit training dataset representation before model training
- ◆ Multi-stakeholder fairness: define different fairness objectives per stakeholder group

■ Potential Challenges & Solutions

Challenge: Fairness Metric Incompatibility

Solution: Demographic parity and equalised odds CANNOT both be satisfied simultaneously when base rates differ between groups (Chouldechova's impossibility theorem) — document this explicitly and let users choose their priority metric

Challenge: Small Subgroup Sample Sizes

Solution: Intersectional subgroups (e.g. elderly Black women) often have <30 samples — confidence intervals become very wide; flag statistically unreliable metrics clearly rather than reporting potentially misleading point estimates

Challenge: Proxy Discrimination

Solution: Removing protected attributes doesn't remove bias if correlated features (zip code as race proxy) remain — add a correlation audit step that flags features with high mutual information to the protected attribute

Challenge: Mitigation Accuracy Tradeoff

Solution: All bias mitigation strategies reduce accuracy to some degree — frame this as a configurable business decision, not a technical failure; let users set their maximum acceptable accuracy drop

■ Learning Resources

- ◆ Fairlearn documentation — Microsoft open-source fairness toolkit
- ◆ AIF360 documentation — IBM AI Fairness 360
- ◆ Chouldechova (2017): Fair prediction with disparate impact (impossibility theorem)
- ◆ Hardt, Price, Srebro (2016): Equality of Opportunity in Supervised Learning
- ◆ EU AI Act technical requirements for high-risk AI systems

◆ COMPAS Recidivism Dataset — ProPublica analysis

■ Project Completion Criteria

- ■ All 5 core fairness metrics implemented with CI and significance tests
- ■ Intersectional subgroup analysis with worst-slice detection functional
- ■ All 4 mitigation strategies (reweighting, resampling, in-processing, post-processing) implemented
- ■ SHAP per-group analysis comparing feature importance across demographics
- ■ PDF audit report generator producing professional multi-section reports
- ■ Streamlit dashboard with model upload, metric charts, and before/after mitigation comparison
- ■ Validated on all 3 benchmark datasets (Adult Income, COMPAS, German Credit)
- ■ Published as pip-installable Python library on GitHub
- ■ Full pytest suite with 80%+ coverage on metric implementations
- ■ README with fairness definitions, API docs, and regulatory context